INFORMACIÓN BÁSICA	
ASIGNATURA:	Tecnología de Objetos
TÍTULO DE LA PRÁCTICA:	Laboratorio 05: Funciones lambda e introducción a Widget App Qt
NÚMERO DE PRÁCTICA:	5 **AÑO LECTIVO:** 2026 **SEMESTRE:** VI
TIPO DE PRÁCTICA:	Grupal (Máximo 2 estudiantes)
INTEGRANTES:	Chipana Jeronimo German Arturo Baca Calsin Leonardo Juan José
DOCENTE:	Edith Giovanna Cano Mamani
REPOSITORIO:	https://github.com/ChipanaGerman/TecnologiaDeObjetos-Lab-B/tree/main/Lab5

I. MARCO GENERAL Y CONCEPTUAL

1. Introducción

El desarrollo moderno de software en C++ ha evolucionado significativamente a través de la adopción de características funcionales integradas al lenguaje. Entre ellas, las funciones anónimas o expresiones lambda permiten especificar lógica de evaluación y predicados directamente en el flujo de ejecución, eliminando la necesidad de declarar funciones aisladas o estructuras con sobrecarga de operadores.

Dentro del ecosistema del framework Qt, el paradigma de Señales y Ranuras (*Signals & Slots*) tradicionalmente requería la declaración explícita de métodos slots en los archivos de cabecera. A partir de las versiones modernas de Qt, la vinculación mediante lambdas permite gestionar respuestas a eventos de interfaces gráficas de manera concisa, modular y manteniendo la verificación de tipos en tiempo de compilación.

2. Objetivos

- **Objetivo General:** Implementar algoritmos de filtrado basados en funciones lambda en C++ estándar y desarrollar interfaces gráficas interactivas en Qt Widgets comunicadas mediante el paradigma de Signals y Slots.
- **Objetivos Específicos:**
  - Construir predicados unarios con captura de entorno para su uso en algoritmos STL (`std::count_if`).
  - Diseñar formularios desacoplados en Qt Designer organizados mediante disposiciones verticales y contenedores adaptables.
  - Conectar señales de widgets visuales (como `QListWidget::itemClicked`) con ranuras de procesamiento y callbacks lambda.

3. Marco Conceptual

- **Expresiones Lambda:** Sintaxis general: `[captura](parámetros) -> tipo_retorno { cuerpo }`. El bloque de captura permite especificar si las variables externas se incorporan por valor (`[=]`), por referencia (`[&]`) o si se captura el puntero a la instancia (`[this]`).
- **Algoritmo `std::count_if`:** Función de orden superior de la librería `<algorithm>` que evalúa un predicado booleano sobre cada elemento de un rango $[inicio, fin)$, retornando la cantidad de coincidencias válidas.
- **Signals & Slots en Qt:** Mecanismo de comunicación débilmente acoplado entre objetos que heredan de `QObject`. Las señales notifican la ocurrencia de eventos y los slots ejecutan la respuesta asociada sin que el emisor requiera conocer los detalles del receptor.

II. DESARROLLO DE LOS EJERCICIOS PROPUESTOS

Ejercicio 1: Conteo de Números Impares y Menores a 20

> ***Enunciado:*** *Contar los números impares y menores de 20 de un vector de números enteros generados de forma aleatoria (la solución puede usar el método `std::count_if`).*

Diseño e Implementación

Para resolver el problema, se genera un vector de enteros empleando el motor de generación pseudoaleatoria `std::mt19937` y la distribución `std::uniform_int_distribution<int>` en el rango $[1, 50]$. Posteriormente, se formula una expresión lambda como predicado unario que evalúa simultáneamente que el valor sea menor estricto a 20 y que su módulo 2 sea impar:

```
auto esImparYMenorQue20 = [](int n) {
    return (n < 20) && (n % 2 != 0);
};
```

El conteo se delega directamente al algoritmo `std::count_if`, recorriendo el vector sin contadores manuales ni bucles tradicionales.

Código Fuente (`Lab5/Ejercicio1/main.cpp`)

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>

using namespace std;

int main() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<int> dis(1, 50);

    const int TAM = 20;
    vector<int> numeros(TAM);

    for (int i = 0; i < TAM; ++i) {
        numeros[i] = dis(gen);
    }

    cout << "Vector generado:" << endl;
    for_each(numeros.begin(), numeros.end(), [](int n) {
        cout << n << " ";
    });
    cout << "\n\n";

    // Condicion: impar y menor que 20
    auto esImparYMenorQue20 = [](int n) {
        return (n < 20) && (n % 2 != 0);
    };

    int total = count_if(numeros.begin(), numeros.end(), esImparYMenorQue20);

    cout << "Numeros impares y menores a 20:" << endl;
    for_each(numeros.begin(), numeros.end(), [&esImparYMenorQue20](int n) {
        if (esImparYMenorQue20(n)) {
            cout << n << " ";
        }
    });
    cout << "\n\n";

    cout << "Total de elementos que cumplen la condicion: " << total << endl;

    return 0;
}
```

Ejecución y Verificación

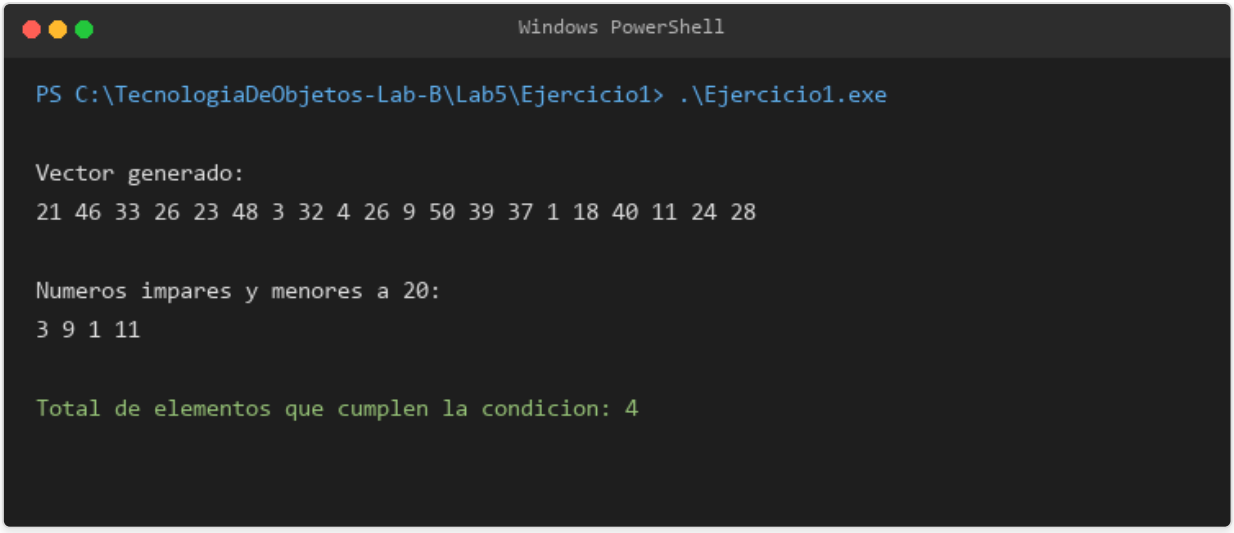


Figura 1: Salida en consola de la ejecución del Ejercicio 1, demostrando la generación aleatoria y el filtrado por predicado lambda.

Ejercicio 3: Interfaz Gráfica con Lista de Países, Capital e Idioma

> ***Enunciado:*** *Crear una interface gráfica (que implemente señales y slots) que muestre una lista de países, al dar clic sobre alguno que se muestre un Label o Text con el idioma y capital correspondiente.*

Diseño de la Interfaz Visual (`mainwindow.ui`)

Se construyó un formulario basado en `QMainWindow` en Qt Designer estructurado verticalmente:

- `lblTitulo` (`QLabel`): Etiqueta superior que orienta al usuario a seleccionar un país.
- `listPaises` (`QListWidget`): Lista scrolleable con los países registrados en el sistema.
- `frameDetalle` (`QFrame`): Contenedor estilizado con borde y esquinas redondeadas que alberga:
  - `lblPais` (`QLabel`): Nombre del país seleccionado con tipografía en negrita.
  - `lblCapital` (`QLabel`): Capital correspondiente.
  - `lblIdioma` (`QLabel`): Idioma oficial.

Manejo de Datos y Conexión Signals & Slots con Lambdas

La información de los países se modela mediante la estructura `InfoPais` y se almacena en un diccionario asociativo `QMap<QString, InfoPais>`. Para garantizar reactividad sin acoplar innecesariamente la UI, se conecta la señal nativa `&QListWidget::itemClicked` con una función lambda que invoca al slot de actualización:

```
connect(ui->listPaises, &QListWidget::itemClicked, this, [this](QListWidgetItem *item) {
    onPaisSeleccionado(item);
});
```

Código Fuente: Archivo de Cabecera (`mainwindow.h`)

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QListWidgetItem>
#include <QMap>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

struct InfoPais {
    QString capital;
    QString idioma;
};

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void onPaisSeleccionado(QListWidgetItem *item);

private:
    Ui::MainWindow *ui;
    QMap<QString, InfoPais> mPaises;

    void cargarPaises();
};

#endif // MAINWINDOW_H
```

Código Fuente: Implementación (`mainwindow.cpp`)

```
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    setWindowTitle("Ejercicio 3 - Informacion de Paises");

    cargarPaises();

    // Conexion con Signal y Slot usando lambda
    connect(ui->listPaises, &QListWidget::itemClicked, this, [this](QListWidgetItem *item) {
        onPaisSeleccionado(item);
    });
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::cargarPaises()
{
    mPaises = {
        {"Peru", {"Lima", "Espanol"}},
        {"Argentina", {"Buenos Aires", "Espanol"}},
        {"Brasil", {"Brasilia", "Portugues"}},
        {"Chile", {"Santiago", "Espanol"}},
        {"Colombia", {"Bogota", "Espanol"}},
        {"Francia", {"Paris", "Frances"}},
        {"Alemania", {"Berlin", "Aleman"}},
        {"Italia", {"Roma", "Italiano"}},
        {"Japon", {"Tokio", "Japones"}},
        {"Canada", {"Ottawa", "Ingles y Frances"}}
    };

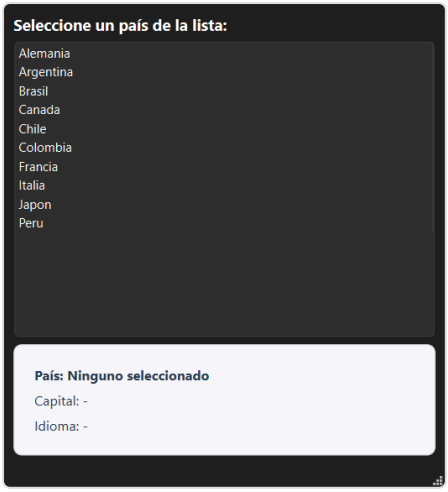
    for (auto it = mPaises.begin(); it != mPaises.end(); ++it) {
        ui->listPaises->addItem(it.key());
    }
}

void MainWindow::onPaisSeleccionado(QListWidgetItem *item)
{
    if (!item) return;

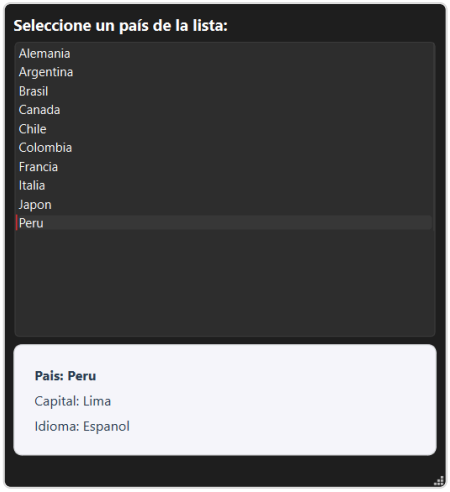
    QString nombrePais = item->text();
    if (mPaises.contains(nombrePais)) {
        InfoPais info = mPaises[nombrePais];
        ui->lblPais->setText("Pais: " + nombrePais);
        ui->lblCapital->setText("Capital: " + info.capital);
        ui->lblIdioma->setText("Idioma: " + info.idioma);
    }
}
```

Pruebas de Funcionamiento de la Aplicación

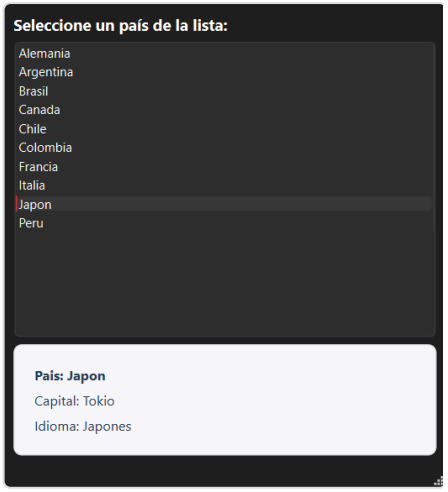
A continuación se muestran las capturas de pantalla de la aplicación ejecutándose en tiempo real, verificando la reactividad ante los clics del usuario:



(a) Estado Inicial: Formulario al iniciar sin selección activa.



(b) Selección 'Perú': Muestra Capital: Lima / Idioma: Español.



(c) Selección 'Japón': Muestra Capital: Tokio / Idioma: Japonés.

III. CONCLUSIONES Y RETROALIMENTACIÓN

Conclusiones

- **Modularidad y Claridad con Expresiones Lambda:** La inclusión de lambdas en C++ simplifica la implementación de lógica puntual evitando la fragmentación del código en funciones externas auxiliares, facilitando el mantenimiento y la comprensión de algoritmos STL.
- **Eficiencia con Algoritmos de Orden Superior:** El uso de `std::count_if` junto con predicados unarios elimina la necesidad de bucles tradicionales y variables contadoras acumulativas, asegurando una programación declarativa menos propensa a errores de límite o desbordamiento.
- **Desacoplamiento Efectivo en Qt:** El sistema de *Signals & Slots* permite que los componentes gráficos notifiquen eventos sin depender directamente de las clases receptoras. La posibilidad de usar lambdas como destinatarias agiliza el desarrollo sin comprometer la seguridad de tipos evaluada durante la compilación.

Retroalimentación General

- **Buenas Prácticas en Signals & Slots:** Se adoptó la sintaxis moderna con punteros a funciones miembro (`&Clase::signal`) en lugar de las macros tradicionales `SIGNAL()` y `SLOT()` de Qt 4, lo que previene fallos silenciosos en tiempo de ejecución.
- **Manejo Estructurado de Datos en GUI:** Se recomienda encapsular los registros de información en estructuras tipadas (como `InfoPais`) y asociarlos mediante tablas o mapas antes que incrustar texto directamente en los eventos gráficos, garantizando escalabilidad hacia orígenes de datos externos o bases de datos SQLite.

IV. REFERENCIAS

- Documentación oficial de Qt Framework: https://doc.qt.io/
- Documentación de referencia C++ STL (`std::count_if`): https://en.cppreference.com/w/cpp/algorithm/count
- Manual de Expresiones Lambda en C++: https://en.cppreference.com/w/cpp/language/lambda
- Manual de referencia de Qt Creator IDE: https://doc.qt.io/qtcreator/